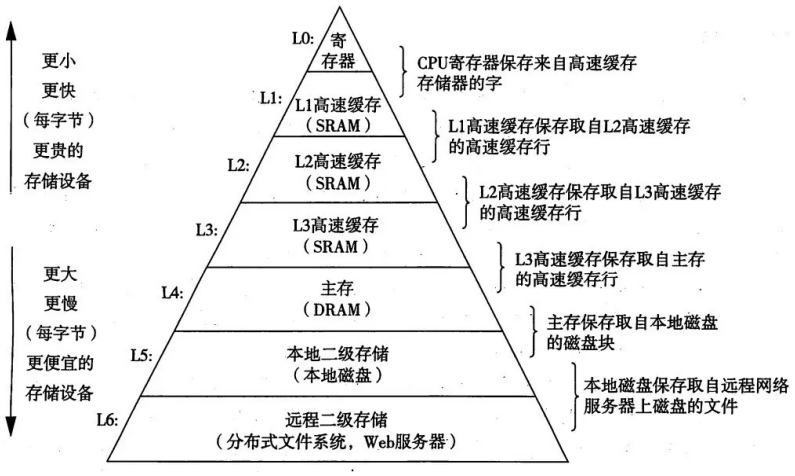
4、内存优化 —— 开发中解决过最难的问题是什么？

内存原理

操作系统内存机制

物理内存



上图就是多层存储器的具体情况，我们**平时常说的内存，实际上就是指的 L4 主存**。而 L1-L3 高速缓存和主存相比，速度更快，并且它们都已经集成在 CPU 芯片内部了。其中 L0 寄存器本身就是 CPU 的组成部分之一，读写速度最快，操作耗费 0 个时钟周期。

CPU寻址

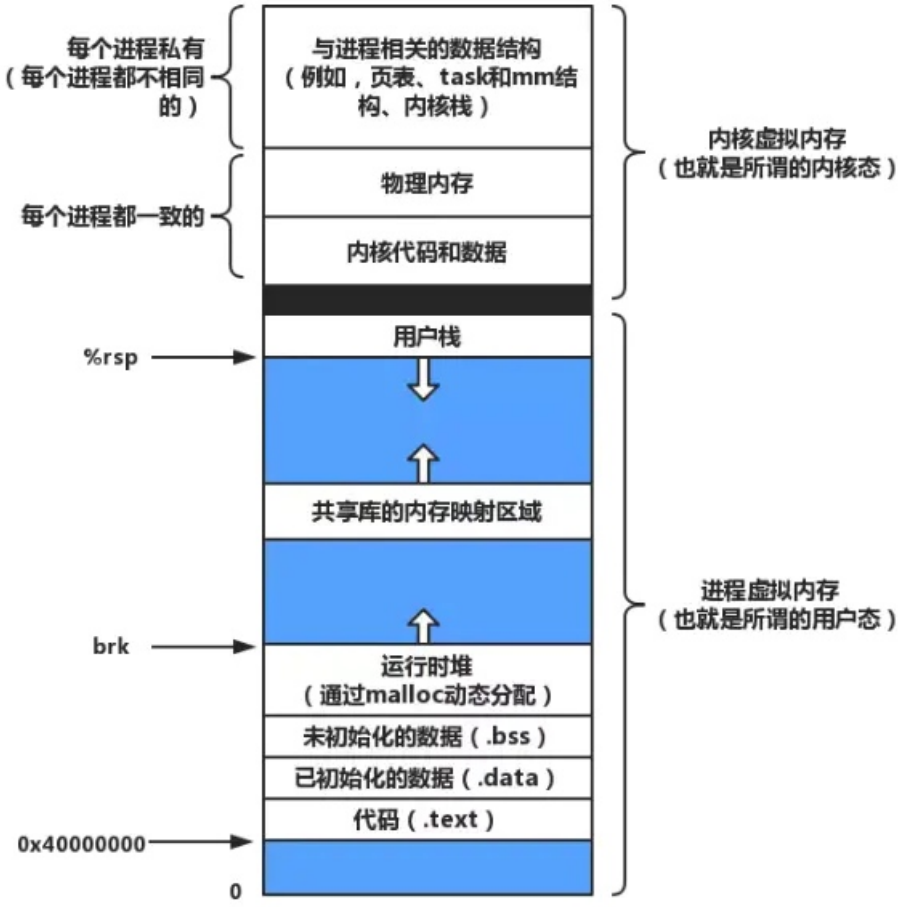
CPU 直接通过物理地址去访问对应的内存，这样也被叫做物理寻址。

因为直接暴露的是物理地址，所以进程可以访问到任何物理地址，用户进程想干嘛就干嘛，这是非常危险的。

现代处理器使用的是虚拟寻址的方式，CPU 通过访问虚拟地址（Virtual Address），经过翻译获得物理地址，才能访问内存。

虚拟内存

刚才提到，直接使用物理寻址，会有地址空间缺乏保护的严重问题。那么如何解决呢？实际上在使用了虚拟寻址之后，由于每次都会进行一个翻译过程，所以可以在翻译中增加一些额外的权限判定，对地址空间进行保护。所以，对于每个进程来说，操作系统可以为其提供一个独立的、私有的、连续的地址空间，这就是所谓的虚拟内存。



虚拟内存最大的意义就是保护了进程的地址空间，使得进程之间不能够越权进行互相地干扰。对于每个进程来说，操作系统通过虚拟内存进行”欺骗”，进程只能够操作被分配的虚拟内存的部分。与此同时，进程可见的虚拟内存是一个连续的地址空间，这样也方便了程序员对内存进行管理。

对于进程来说，它的可见部分只有分配给它的虚拟内存，而虚拟内存实际上可能映射到物理内存以及硬盘的任何区域。由于硬盘读写速度并不如内存快，所以操作系统会优先使用物理内存空间，但是当物理内存空间不够时，就会将部分内存数据交换到硬盘上去存储，这就是所谓的 Swap 内存交换机制。有了内存交换机制以后，相比起物理寻址，虚拟内存实际上利用硬盘空间拓展了内存空间。

总结起来，虚拟内存有下面几个意义：保护了每个进程的地址空间、简化了内存管理、利用硬盘空间拓展了内存空间。

内存分页

为了方便映射和管理，虚拟内存和物理内存都被分割成相同大小的单位，物理内存的最小单位被称为帧（Frame），而虚拟内存的最小单位被称为页（Page）。

注意**页和帧大小相同**。

内存分页最大的意义在于，支持了物理内存的离散使用。由于存在映射过程，所以虚拟内存对应的物理内存可以任意存放，这样就方便了操作系统对物理内存的管理，也能够可以最大化利用物理内存。

iOS内存机制

- 使用了虚拟内存

- 内存有限，但是单应用可用内存大
- 没有内存交换机制

> **注意**：内存警告和OOM都是针对的物理内存

内存使用相关数据的计算，通过**task_vm_info_data_t**就可以：

代码块

```
/// 返回使用百分比
    static func memoryUsage() -> MemoryUsageData {
        var taskInfo = task_vm_info_data_t()
        var count = mach_msg_type_number_t(MemoryLayout<task_vm_info>.size) / 4
        let result: kern_return_t = withUnsafeMutablePointer(to: &taskInfo) {
            $0.withMemoryRebound(to: integer_t.self, capacity: 1) {
                task_info(mach_task_self_, task_flavor_t(TASK_VM_INFO), $0, &count)
            }
        }

        var used: UInt64 = 0
        if result == KERN_SUCCESS {
            used = UInt64(taskInfo.phys_footprint)
        }

        let total = ProcessInfo.processInfo.physicalMemory

        let bytesInMegabyte = 1024.0 * 1024.0
        let usedMemory = Double(used) / bytesInMegabyte
        let totalMemory = Double(total) / bytesInMegabyte
        let finalUserdData = usedMemory / totalMemory
        let finalUserdStr = String(format: "%.1f of %.0f MB", usedMemory, totalMemory)
        return MemoryUsageData(data: finalUserdData, detail: finalUserdStr, usedMemory: usedMemory, totalMemory: totalMemory)
    }
```

Jetsam机制

iOS对内存警告、OOM崩溃的处理机制就是**Jetsam机制**。Jetsam会始终监控系统整体的内存使用情况，当内存不足时会根据优先级、内存占用大小杀掉一些进程，并记录成JetsamEvent。

Jetsam维护了一个优先级队列。

首先，内核有一个内核优先级最高的线程。这个线程会维护两个列表，一个是基于优先级的进程列表，另一个是基于每个进程消耗的内存页的列表。

其次，这个线程会监听内核“pageout线程”对整体内存使用情况的通知，在内存告急时，向每一个进程转发内存警告，也就是didReceiveMemoryWaring

而触发OOM时，杀掉进程的方式分为同步或异步两种：同步的话会立刻杀掉进程，当然，这里会进行优先级判断，杀掉优先级最低的进程；同一优先级再根据内存使用大小，杀掉内存占用大的进程。异步的话，只会标记当前进程，然后通过专门的内存管理线程去杀死进行。

> 正常来说，处于前台的进程优先级高于后台进程。所以通常后台进程容易被kill

内存警告

当内存不够用时，iOS 的处理是会发出内存警告，告知进程去清理自己的内存。iOS 上一个进程就对应一个 app。代码中的 `didReceiveMemoryWarning()` 方法就是在内存警告发生时被触发，app 应该去清理一些不必要的内存，来释放一定的空间。

> 注意：并非所有内存警告都是由App造成的，例如在内存较小的设备上，当你接听电话时，也可能发生内存警告。

内存警告消息来自主线程，所以应避免主线程这时候卡顿或者分配过大内存或者快速分配。

如果app因为内存警告被kill，会生成LowMemoryxx.log日志

OOM 崩溃

如果 app 在发生了内存警告，并进行了清理之后，物理内存还是不够用了，那么就会发生 OOM 崩溃。

单个 app 可用内存达到达到了设备可用内存 55%。当 app 使用的内存超过这个临界值，就会发生 OOM 崩溃。可以看出，单个 app 的可用物理内存实际上还是很大的，要发生 OOM 崩溃，**绝大多数情况下都是程序本身出了问题**。

FOOM归因

什么样的崩溃才是FOOM崩溃?

FOOM归因方案

内存分类

从内存使用状态的角度，内存可以分为：

- Clean Memory：未使用的内存
- Dirty Memory：写入过数据的内存
- Compressed Memory：压缩后的内存

> 这里稍微解释一下：iOS内存吃紧的时候，系统会将不使用的内存进行压缩，直到下一次访问的时候进行解压。
>
> 这里不使用的内存，可能是指非活跃的应用所占用的内存。

> 当内存从压缩解压时，也可能因为一瞬间的内存使用放大，而导致OOM。

从内存管理的角度，内存可以分为：

- Leaked Memory：内存泄露。未能正常释放掉的内存
- Abandoned Memory：被遗弃的内存。
- Cached Memory：缓存内存。

> 其中Leaked memory和abandoned memory都属于应该释放而没有释放的内存，都属于内存泄露。

从Allocation的数据看，主要的类型：

- CG raster data:光栅数据，即为UIImage的解码数据; 一般是远程图片数据
- CoreAnimation: UIView/CALayer
- Image IO: 本地图片数据
- ... ...

内存检测与分析

内存检测工具调研整理

Xcode本身检测工具集：

Xcode memory gauge：

能够实时看到内存使用情况。

其实是一个非常好的内存使用工具，因为它非常实时。像instrument，一跑可能就会卡顿，所以对于那种一瞬间的内存爆炸问题，Instrument是无从下手的。

Xcode debug memory Graph：

能够查看到对象之间的引用依赖关系。对于没有及时释放的对象检测，有一定助益。

然后这个也可以及时导出xx.memgraph文件，然后结合一些vmmap、leaks、heap命令打印出一些进程信息。

代码块

```
$vmmap xx.memgraph // 查看详细报告，比如虚拟内存、实际内存、dirty memory等
```

```
$leaks xx.memgraph // 查看内存泄露
```

代码块

```
$heap xx.memgraph // 查看堆区对象的使用
```

Instrument检测工具集：

Instrument - Allocations：

主要用于查看对象创建

比如使用generation mark标记，来对比两个mark之间的对象差异，从而进行分析。

Instrument - Leaks：

主要用于查看内存泄露

手机设备：

如果在手机上看到了崩溃，则可以去设置——隐私——分析与改进中，查看到崩溃（OOM）的日志。然后导出文件进行分析。（崩溃也适用）

第三方开源库：

iOS内存管控实战（中）-分析工具篇

MLeaksFinder

MLeaksFinder是一个轻量级的、无侵入式的内存泄露检测工具，只需要引入库。就可以自动在App运行过程中，检测到内存泄露的对象并弹窗提醒。但是它只检测UIView和UIViewController对象的内存泄露。其他内存需要额外扩展。

由于MLeaksFinder的便捷使用，所以线下使用是非常方便的。

原理简介：

为基类 NSObject 添加一个方法 `-willDealloc` 方法，该方法的作用是，先用一个弱指针指向self，并在一小段时间(3秒)后，通过这个弱指针调用 `-assertNotDealloc` ，而 `-assertNotDealloc` 主要作用是直接中断言。

代码块

```
(BOOL)willDealloc {
  ...
  __weak id weakSelf = self;
```

```
  dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(2 * NSEC_PER_SEC)), dispatch_get_main_queue(), ^{
      __strong id strongSelf = weakSelf;
      [strongSelf assertNotDealloc];
  });
  return YES;
}
(void)assertNotDealloc {
  ...
  NSLog(@"Possibly Memory Leak.\nIn case that %@ should not be dealloced, override -willDealloc in %@ by returning NO.\nView-ViewController stack: %@", className, className, [self viewStack]);
}
```

劣势：

- 一些特殊情况，不会被释放的对象（比如单例），需要重载willDealloc，return no。否则也会弹窗。
- 特殊情况，比如语音房悬浮窗，也会弹窗。

FBRetainCycleDetector

Facebook开源的用于检查循环引用的库。

使用：

代码块

```
@implementation ViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.

    _handlerBlock = ^{
        NSLog(@"%@", self);
    };

    FBRetainCycleDetector *detector = [FBRetainCycleDetector new];
    [detector addCandidate:self];
    NSSet *retainCycles = [detector findRetainCycles];
    NSLog(@"%@", retainCycles);
}

@end
```

原理简介：

从传入的目标对象开始查找其强引用的对象列表，在这个强引用对象列表里继续查找强引用对象，默认查找 10 层。最后在整个有向图中应用DFS算法查找环。如果有向图存在环，则说明目标对象存在循环引用。

所以，当你怀疑某个对象有循环引用时，可以直接使用这个库，将对象传入进行查找。

当然，**劣势**就是不方便对整个项目的循环引用进行排查，而且只能在OC上有效。

可以学习的点：

普通OC对象，可以通过哪些方式强引用其他对象？

- ivar成员变量：__strongIvar
- 集合中的元素：NSArray
- 关联对象

当前这些并一定都是强引用，也可能是弱引用。需要底层逐个处理。

内存优化实战：

内存泄露：

就像上文说到的，出现OOM的case，大都是由于程序本身造成的。而程序当中，内存泄露就是我们经常碰到的问题。它会导致内存一直累积，无法及时释放。随着app使用时长越来越长，可能就会收到内存警告和OOM

而内存泄露基本都是由于对象之间的**循环引用**引起的。

常见的循环引用有：

- block里强引用了self，需要使用weak 或 unowned进行处理
- delegate没有使用weak
- NSTimer 的使用

对于内存泄露，通常可以使用MLeaksFinder、Instrument-Leaks、Xcode debug memory Graph等进行检测排查。

UIWebView和WKWebView内存问题

UIWebView 会因为内存使用过大而崩溃，WKWebView 苹果进行了优化，不会 Crash 但会导致白屏，不显示内容。

解决方法是监听到 URL 为 nil 或者接收到 WKNavigationDelegate 的 `webViewWebContentProcessDidTerminate` 时，reload 页面。

代码块

```swift
extension TPVirtualPortraitViewController: WKNavigationDelegate {
    func webViewWebContentProcessDidTerminate(_ webView: WKWebView) {
        webView.reload()
    }
}
```

图片资源处理

Image 图片加载优化

UIImageView加载图片大概流程：

- UIImage与UIImageView的关系，就类似于Model与View的关系。UIImage负责加载图片内容，UIImageView负责显示和渲染它。
- 图片加载的流程：**加载——解码——渲染。**
  - **加载：**首先都是通过图片名称从Asset或者Bundle或者磁盘当中查找图片，找到图片后加载进UIImage里面。
  - **解码：**图片在加载进UIImage后，不会立即进行解码操作，而是在图片第一次显示到屏幕上时，才调用内部的解码方法进行解码操作。解码就是将不同压缩格式的图片转码成图片的原始像素数据的过程。
  - **渲染：**解码完成后，图片才会被赋值给UIImageView的图层上，最终渲染到手机屏幕上。
- **Data Buffer**：数据缓冲区。保存了图像压缩格式时的图像数据。比如png、jpeg的图片。

  Data Buffer是图片原始数据，可以理解为PNG、JPEG格式的图片，这种数据包含图片的拍摄时间、地点、所以不能直接用来描述图像像素信息。Data Buffer的大小其实就是磁盘里存储的图片大小。
  - png：png是图片无损压缩格式，支持alpha通道；png图片加载会比JPEG更长，因为文件可能更大，但是解码会相对更快，而且xcode会把png图片进行解码优化之后引入工程。
  - jpeg：jpeg是图片有损压缩格式，可以指定0~100的压缩比；jpeg图片更小，加载更快，但是jpeg解压算法更加复杂，所以解压要慢很多。
  - 矢量图：矢量图是通过类似xml格式对路径和填充颜色进行渲染描述的，所以它的大小比位图小，且无论放大多少都不易失真。
- **Image Buffer**：图像缓冲区。表示图像在内存中的表示，缓冲区的每一个元素，描述了图像中每个像素的颜色和透明度。

  Image Buffer存储的就是图片解码后的像素数据，也就是我们常说的位图。也可以说**解码就是从data buffer生成Image Buffer的过程**。Image Buffer的大小与图片本身的大小成正比。
  - 图片大小计算：**像素宽 * 像素高 * 4** （单位byte）
- **Frame Buffer**：帧缓冲区。保存了图像实际渲染后的输出。

大图处理：

首先，图片太大，解码时候所需要的内存就会非常大。所以对于大图，通常要考虑的做法就是：

不缓存

- 本地图片图片读取：

  imageNamed 会被缓存到内存中，适用于频繁使用的小图片；imageWithContentOfFile 适用于大图片，持有者生命周期结束后既被释放。

- 远程大图不进行缓存设置（这些设置在新版本中，貌似无法使用了）：

  因为SDWebImage，这种是会统计解码后，缓存起来的。所以对于大图，需要额外处理。

  代码块

  ```
  kCGImageSourceShouldCache：避免缓存解码后的数据，因为这个是缩略图，之后的使用场景可能就不一样，所以不要做缓存
  kCGImageSourceShouldCacheImmediately：YES表示立马解码，而不是等到渲染的时候才解码
  ```

  图片缓存与SDWebImage源码详解

  SDWebImage 加载大量图片崩溃

降低采样率：

说白了就是使用**缩略图**。因为上面说到，解码时，Image Buffer的大小与图片本身成正比。所以对于图片比较大，ImageView比较小的场景，可以使用降低采样率的方式。

SDWebImage这种第3方库，已经实现了降低采样率的做法。

超大图使用特殊图层CATiledLayer：

- 如果是那种像微信、微博详情那样的大图，则应该全屏加载大图，通过拖动来查看不同位置图片的细节。技术细节就是使用苹果的**CATiledLayer**去加载，它**可以分片渲染，滑动时通过映射原图指定位置的部分图片数据解码渲染**。

使用替代方案：

- 对于动图，可以使用MP4进行代替。
- HEIC 格式：画质比 JPEG 节省 50% 内存
- Webp、pag动图：TODO
- 火山引擎的自研框架：https://developer.volcengine.com/articles/7238813685727100965

动图优化

https://blog.csdn.net/u014600626/article/details/138756783

缓存的使用：

- 缓存分级

  分级的好处，主要是为了及时清理内存。毕竟有的场景是需要对小图片进行缓存的。

  这里主要是说远程图片的缓存。比如设置SDWebImageCache

代码块

```
/// 单独用于语音房缓存
/// 主要是退出时及时清理
extension TPAudioLiveImageCache {
    static var audioLiveImageCache: TPAudioLiveImageCache = {
        let cache = TPAudioLiveImageCache(namespace: "audioLive", diskCacheDirectory: TPGravityDocumentFolder.data.path.stringByAppendingPathComponent("audioLiveCache"))
        cache.config.maxDiskAge = 60 * 60 * 24 * 7 // 一周
        cache.config.maxMemoryCost = 1024 * 1024 * 100 // 100MB
        cache.config.maxMemoryCount = 300 // 图片数量
        cache.config.maxDiskSize = 1024 * 1024 * 200
        return cache
    }()
}
```

- 及时清理图片缓存
  - 对不需要使用时的缓存及时清理。比如每次最新Feed列表下拉刷新时，及时清理掉之前缓存的图片。
  - 退出语音房房，及时清理掉语音房内的图片缓存。

内存监控预警

- 远程图片解码前预警：SD网络图片解码前内存预警
- 对App内本地资源进行梳理，排查内存过大图片，比如大于2M。
- 页面内存预警：在VC创建和销毁时，分别主动获取内存使用大小，进行上报。
- 内存使用打点上报。

其他实战案例

1. 使用Instrument-Allocation处理**大量对象创建的内存问题**

例子：在执行ScrollView滚动的回调中，调用了刷新导航栏暗黑模式图片切换的函数。函数中多个UIButton在执行setImage函数，（内部的imageNamed是有缓存的）

所以在快速滑动的过程中，内存一张在涨，导致卡顿和OOM。

解决方案：添加标志位，确保函数只执行一次。

经验：Allocation 的 generation mark非常适合这种不断创建对象的检测

2. 使用Xcode memory gauge 处理**内存激增问题**

例子：滑动feed的时候，偶现内存急剧爆增的情况。后面定位到，是在创建MutableAttributeString时，在没有设置大小的情况下，设置yy_aligment。这种情况可能会导致临时生成一个非常大的rect（参考库issues）

解决方案：去掉yy_aligment设置

经验：这种一瞬间的内存激增，不适合用Instrument，因为很难复现这个现象。可以使用**注释二分法**慢慢定位。

3. **使用MLeaksFinder + FBRetainCycleDetector捕获内存泄露问题**

例子：主要是忘记写weak 的 block

其他

- **用 UIGraphicsImageRenderer 代替 UIGraphicsBeginImageContextWithOptions**

参考：

深入探索 iOS 内存优化 - 掘金

内存检测工具调研整理

iOS Memory 内存详解 | 若然何如

MLeaksFinder：精准 iOS 内存泄露检测工具

GitHub - Zepo/MLeaksFinder: Find memory leaks in your iOS app at develop time.

[UIImage imageNamed:] 缓存策略窥探

iOS-UIImage imageWithContentsOfFile 和 imageName 对照(图片加载的内存及优化)_uiimage(contentsoffile-CSDN博客

iOS 图片加载解码造成内存峰值的一些思考

Instruments学习之Allocations分析内存 - 掘金

Instruments - Leaks的使用 - 掘金

【翻译】如何使用 Xcode Memory Graph Debugger 检测 iOS 内存泄漏和循环引用

Debug Memory Graph检查内存泄漏

ios统计内存使用量 ios怎么看内存占用情况_mob6454cc6b8546的技术博客_51CTO博客

SDWebImage内存爆增问题及Image解压缩 - 掘金

FBRetainCycleDetector 源码笔记

iOS内存管控实战（上）—原理篇

关于iOS内存的深入排查和优化 - 掘金